

$+$

Gemini 3.5 Flash • Ran for 94s

⚡ Strategic Choice Frameworks

- **Pros & Cons List:** Evaluates single binary decisions, presenting factors with numerical importance weights (1-5) side-by-side, paired with a visual **Balance Bar** comparing cumulative impact.
- **Comparative Table (Matrix):** Formulates custom evaluation criteria dynamically (up to 5 options), rating each option from 1 (poor) to 5 (excellent) with brief, AI-powered justifications and total cumulative score counters.
- **SWOT Analysis Strategic Quad:** Constructs a comprehensive four-quadrant matrix (Strengths, Weaknesses, Opportunities, Threats) to analyze major personal or organizational moves.

The Decisive AI Engine & Closure Flows

- **The Tiebreaker Verdict:** Every analysis finishes with a custom-crafted, definitive advice card that breaks hesitation. The AI acts as an active mentor, deliberately recommending a single, high-conviction path forward rather than saying "it depends."
- **Actionable Choice Lock-in:** Users can lock in their final course of action directly under the verdict (either following the AI's recommendation or forging their own path).
- **Decisiveness Index & Logs:** A persistent sidebar keeps a chronological history of all scenarios (saved securely in your local browser cache), charting your custom **Trust Factor** percentage based on how often you followed the recommended choice.
- **Sample Dilemma Presets:** Includes clickable presets (e.g., purchasing an EV, choosing an anniversary trip, launching a side project) to let you explore the tool instantly.

↪ Restore

Fix

×



- Code

▼

assets

src

- components

DecisionFo...

HistorySide...

ResultView....

App.tsx

index.css

main.tsx

types.ts

 .env.example

 .gitignore

> index.html

```
} metadata.json
```

```
} package.json
```

```
server.ts
```

```
} tsconfig.json
```

vite.config.ts


```

17 export default function App() {
18   const onSubmit = async () => {
19     if (!res.ok || !result.success) {
20       throw new Error(result.error || "Failed to retrieve analysis from The Tiebreaker.");
21     }
22
23     const newId = `dec-${Date.now()}`;
24     const newDecision: DecisionResult = {
25       id: newId,
26       title: formData.decisionTitle,
27       createdAt: new Date().toISOString(),
28       type: formData.analysisType,
29       additionalContext: formData.additionalContext,
30       rawResponse: result.data,
31     };
32
33     const updatedHistory = [newDecision, ...history];
34     saveHistory(updatedHistory);
35     setActiveId(newId);
36     showToast("Dilemma analyzed successfully! Tiebreaker verdict delivered.");
37   } catch (err: any) {
38     console.error("Error submitting decision:", err);
39     setError(err.message || "An unexpected error occurred. Please try again.");
40   } finally {
41     setIsLoading(false);
42   }
43 };
44
45 const handleLockChoice = (choice: string, followedAI: boolean) => {
46   const updatedHistory = history.map((item) => {
47     if (item.id === activeId) {
48       return {
49         ...item,
50         userFinalChoice: choice,
51         followedAI,
52       };
53     }
54     return item;
55   });
56   saveHistory(updatedHistory);
57   showToast(`Decision locked in: "${choice}"`);
58 };
59
60 const handleShare = () => {
61   const activeResult = history.find((h) => h.id === activeId);
62   if (!activeResult) return;
63
64   try {
65     const formattedText = `The Tiebreaker Analysis: "${activeResult.title}"\nMethod: ${activeResult.type.toUpperCase()}\nVerdict: \n

```